Governance-Aware Provenance for Privacy Auditing in Machine Learning Pipelines

No Author Given

No Institute Given

Abstract. abstract here

Keywords: keywords here

1 Introduction

Machine-learning (ML) systems are increasingly deployed in domains involving sensitive personal data, including healthcare, finance, and public administration. Modern ML development relies on complex pipelines composed of data curation, preprocessing, feature engineering, integration, aggregation, and model training steps. While such pipelines enable reproducibility and scalability, they also introduce significant privacy governance challenges.

Privacy risk in ML pipelines does not arise solely from the presence of identifiable attributes at input. Instead, exposure evolves throughout execution as data are transformed, combined, or summarised. Feature engineering may preserve sensitive information even when explicit identifiers are removed. Dataset fusion can introduce re-identification risks that were absent in individual sources. Learning algorithms may embed information originating from training data into derived models. Conversely, aggregation may attenuate identification risk through summarisation. Understanding privacy exposure therefore requires reasoning about how information propagates across executed transformations.

Regulatory frameworks such as the General Data Protection Regulation (GDPR) further require organisations to demonstrate accountability throughout the data lifecycle. Auditing tasks include determining which downstream artifacts depend on a deleted record, verifying that processing respects authorised purposes, identifying transformations responsible for introducing privacy risks, and assessing whether sensitive information unnecessarily influenced model production. These questions extend beyond static data inspection and require reasoning about execution history.

Provenance information provides a natural foundation for addressing these challenges. Retrospective provenance records how artifacts are produced through concrete executions, capturing dependencies between data, transformations, and derived artifacts. However, provenance alone captures influence relationships without representing privacy semantics or governance obligations.

Motivated with the above observation, we proposes, in this paper, a provenance-based framework for privacy-aware auditing of machine-learning pipelines. We

extend life-cycle provenance with a privacy ontology capturing sensitivity categories, authorised usage contexts, erasure obligations, and organisational access responsibilities. Privacy properties evolve through execution using operator-aware propagation semantics that account for contributor granularity and transformation behaviour. Finally, auditing queries are evaluated relative to organisational roles, enabling governance-aware reasoning over authorised provenance views. We note here that granularity reasoning applies primarily to structured datasets (dataset, tuple, cell), whereas derived artifacts such as trained models are treated as atomic provenance entities. Our contributions are as follows:

- We introduce a lifecycle provenance model for ML pipelines supporting dataset-, record-, and cell-level reasoning across executions.
- We propose a privacy ontology fragment that associates provenance entities with sensitivity categories, usage constraints, erasure obligations, and authorised organisational roles.
- We define an operator-aware propagation algebra capturing contributor-based combination, policy-driven minting of new privacy risks, and attenuation through aggregation.
- We formalise role-aware auditing queries combining contribution-based reasoning and exposure-based analysis over authorised provenance views.

Together, these mechanisms support systematic privacy auditing across the machine-learning lifecycle while aligning execution analysis with organisational governance requirements. Accordingly, the remainder of the paper is organised as follows. Section 2 discusses privacy exposure in ML pipelines. Section 3 introduces lifecycle provenance foundations. Section 4 presents the privacy ontology. Section 5 defines operator-aware annotation propagation. Section 6 introduces privacy auditing queries. Section ...

2 Privacy Exposure in Machine-Learning Pipelines

ML pipelines transform data through sequences of preprocessing, feature engineering, integration, aggregation, and model training steps. While these transformations enable predictive performance and reproducibility, they also introduce complex privacy risks that extend beyond direct use of sensitive attributes.

Privacy exposure in ML pipelines does not arise solely from the presence of identifiable data at input. Instead, exposure evolves throughout execution as data are transformed, combined, or summarised. Understanding these effects requires reasoning about how information flows across pipeline executions and how privacy properties emerge from transformation semantics.

2.1 Sources of Privacy Exposure

Privacy exposure may arise through multiple mechanisms during pipeline execution.

Direct sensitivity propagation. Sensitive attributes such as biometric or health measurements may propagate through intermediate artifacts even when explicit identifiers are removed. For example, engineered features derived from heart-rate measurements may continue to encode biometric information despite schema transformations.

Risk introduced by data combination. Combining datasets originating from different sources may introduce risks not present in individual inputs. Joining clinical data with demographic information using quasi-identifiers can increase linkage potential and enable re-identification. Such risks emerge from transformation semantics rather than from individual contributors alone.

Model-derived exposure. Learning algorithms may embed information originating from training data into trained models or evaluation artifacts. Even when models do not explicitly expose raw records, they may remain influenced by sensitive inputs through memorisation effects or statistical dependence.

Exposure reduction through summarisation. Conversely, certain transformations may reduce identifiability. Aggregation or abstraction steps summarising large populations may attenuate identifier-related exposure by reducing the ability to associate information with individuals.

These examples illustrate that privacy exposure depends not only on data presence but also on how transformations manipulate information during execution.

2.2 Governance Constraints Beyond Sensitivity

Privacy governance extends beyond sensitivity classification alone. Regulatory and organisational requirements impose additional constraints on data usage and lifecycle management.

Purpose limitation. Data subjects may authorise processing only for specific purposes. For example, a patient may consent to clinical research but not commercial product development. Derived artifacts must therefore respect authorised usage contexts inherited from contributing data.

Retention and erasure obligations. Deletion requests or retention policies may require recomputation or removal of downstream artifacts influenced by specific contributors. Identifying affected artifacts requires tracing dependencies across pipeline executions.

Organisational responsibilities. Different organisational actors participate in pipeline governance, including data scientists, auditors, and data protection officers. Privacy auditing tasks may therefore require distinct levels of detail or visibility depending on organisational responsibility.

These governance dimensions interact with transformation semantics and motivate systematic reasoning across lifecycle provenance.

2.3 Auditing Requirements

Supporting privacy governance in ML pipelines requires answering several classes of auditing questions:

- Which downstream artifacts depend on a specific record subject to deletion?
- Does data usage across pipeline executions respect authorised purposes?
- Which transformations introduced or propagated privacy risks?
- Did sensitive information influence model production and therefore warrant further data minimisation analysis?

Answering these questions requires combining dependency reasoning over executed pipelines with privacy-aware interpretation of derived artifacts. The following section introduces lifecycle provenance as a foundation for capturing execution dependencies at multiple granularities.

3 Provenance in ML Pipelines

We distinguish between *prospective* and *retrospective* provenance for ML pipelines. Prospective provenance describes the intended structure of a pipeline prior to execution (e.g., steps and dataflow). Retrospective provenance captures the concrete artifacts and executions observed at runtime and forms the basis for auditing queries in Section 6.

3.1 Prospective Lineage

Prospective lineage represents a pipeline specification as a directed dataflow of steps.

Definition 1 (Pipeline). *A* pipeline *is a tuple $P = (Steps, F)$, where Steps is a finite set of steps and $F \subseteq Steps \times Steps$ is a dataflow relation indicating that outputs of one step may serve as inputs to another.*

Definition 2 (Step). *A* step *is a tuple $s = (I, O, \tau)$, where I and O denote input and output artifact* specifications *(types and schemas), and τ denotes a transformation operator.*

Transformation operators include (among others) filtering and projection, feature construction, aggregation, dataset fusion (e.g., joins), and model training/evaluation. Pipeline specifications are often organised into higher-level phases.

Definition 3 (Stage). *A* stage *is a subset $\Sigma \subseteq Steps$ representing a logical phase of a pipeline (e.g., data curation, learning data preparation, or model learning).*

3.2 Artifacts and Granularity

Pipeline steps consume and produce artifacts such as datasets, intermediate feature tables, and trained models. In many governance settings, auditing requires reasoning at multiple granularities.

Definition 4 (Artifact Specification). *An* artifact specification *is a tuple* $spec = (id, type, schema)$*, where id uniquely identifies the specification, type denotes its category (e.g., dataset, model), and schema describes its structure.*

Definition 5 (Artifact Granularity). *Artifact granularity applies to structured data artifacts. Datasets are decomposed into records (e.g., tuples or subject-level slices), and records into cells representing attribute or feature values. Other artifact types, such as trained models or evaluation reports, are treated as atomic entities without internal containment structure.*

For example, a curated physiological dataset is represented as a dataset entity; Bob's monitoring window corresponds to a record entity; and an individual heart-rate value corresponds to a cell entity.

3.3 Retrospective Lineage

Retrospective lineage captures the concrete executions of prospective specifications, including the materialised artifacts and step executions. We align with W3C PROV [**?**] by representing executed transformations as `prov:Activity` instances and materialised artifacts as `prov:Entity` instances.

Definition 6 (Pipeline Run). *Given a pipeline* P*, a* pipeline run *is a tuple* $PR = (P, id, M)$*, where id uniquely identifies the execution and* M *denotes execution metadata (e.g., timestamps, parameters, environment).*

Definition 7 (Step Run). *Given a step* $s \in Steps$ *and a pipeline run* PR*, a* step run *is a tuple* $SR = (s, PR, id)$*, where id uniquely identifies the execution instance.*

Definition 8 (Artifact Instance). *An* artifact instance *is a tuple* $inst = (id, type, v)$*, where id uniquely identifies the instance, type denotes its category (e.g., dataset/record/cell/model), and* v *denotes the materialised version (e.g., a concrete dataset snapshot or trained model).*

3.4 Execution Dependencies

Dependencies between executions and artifacts are captured using standard PROV relations [**?**].

Definition 9 (Dependency Relations). *Given a step run* SR *and artifact instances* x *and* y*:*

- $used(SR, x)$ *indicates that* SR *consumed* x,
- $wasGeneratedBy(x, SR)$ *indicates that* x *was produced by* SR,
- $wasDerivedFrom(x, y)$ *indicates that* x *was derived from* y.

These relations form a directed provenance graph $G = (V, E)$ over entities and activities, capturing lifecycle dependencies across pipeline stages.

Example 1. A feature-engineering step run may consume a curated dataset instance (containing Bob's record) and generate a derived dataset instance subsequently used for model training.

3.5 Containment and Contribution

To support multi-granularity reasoning, we additionally represent containment between artifact instances: datasets contain records, and records contain cells (cf. Figure 1). We write $containsDR(d, r)$ when a dataset instance d contains a record instance r, and $containsRC(r, c)$ when a record instance r contains a cell instance c. Containment supports top-down policy interpretation (dataset-level declarations applied to records/cells) and bottom-up summarisation (record/cell properties aggregated at dataset level), as formalised in Section 4.

Many auditing tasks require determining which artifacts influence downstream results.

Definition 10 (Contribution). *An artifact instance* x contributes *to an artifact instance* y *if there exists a path of* `wasDerivedFrom` *relations from* y *to* x *in the provenance graph.*

Contribution captures indirect dependencies across multiple transformations and lifecycle stages. Section 4 augments this provenance representation with privacy annotations attached to artifact instances, and Section 5 specifies how such annotations evolve during execution.

4 Privacy Ontology and Exposure

This section introduces an ontology fragment that augments lifecycle provenance with privacy concepts. Figure 1 summarises the main classes and relationships. The fragment associates privacy information with artifact instances while remaining compatible with standard provenance representations.

4.1 Core Concepts

Privacy information is represented through `PrivacyAnnotation` objects linked to provenance entities via the relation `annotates`. Each annotation captures four complementary facets shown in Figure 1:

- sensitivity categories (`hasCategory`),

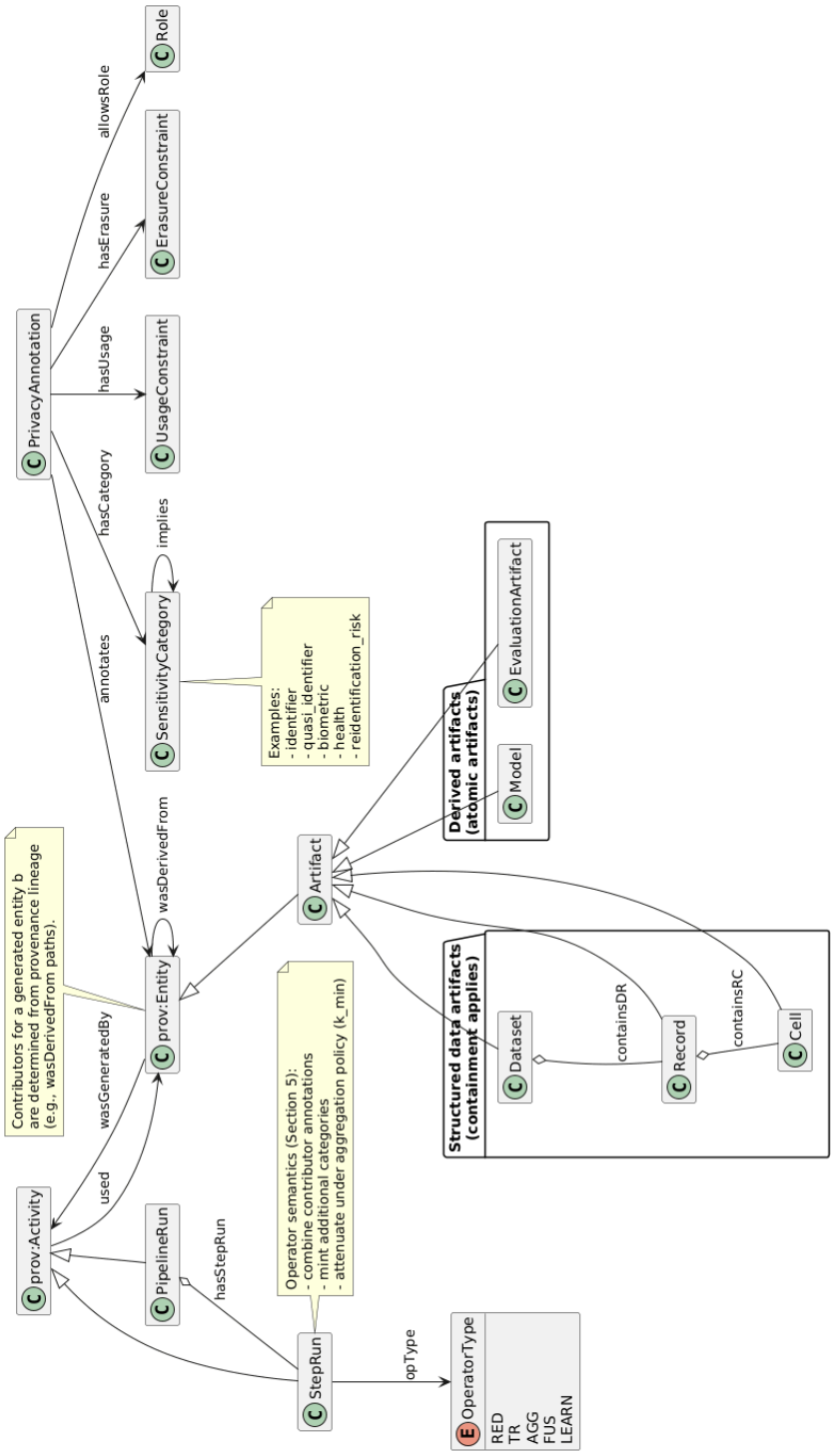

Fig. 1: Privacy ontology fragment extending lifecycle provenance.

- authorised usage contexts (`hasUsage`),
- retention or erasure obligations (`hasErasure`),
- authorised access roles (`allowsRole`).

Sensitivity categories capture data protection notions such as *identifier*, *biometric*, *health*, or *reidentification_risk*. Usage constraints represent permitted purposes under which data may be processed (e.g., *clinical research*). Erasure constraints express deletion obligations, for instance those triggered by a data subject request or retention policy. Finally, access roles represent organisational responsibilities or privileges associated with governance processes, such as *DataScientist*, *Auditor*, *DataSteward*, or *DPO*. Access roles enable role-aware auditing and restrict the visibility of provenance information during query answering.

Example. Let c be a cell entity representing Bob's heart-rate value. A privacy annotation pa may satisfy $annotates(pa, c)$, $hasCategory(pa, biometric)$, and $hasUsage(pa, clinical\ research)$. If Bob requests deletion, the annotation additionally satisfies $hasErasure(pa, delete)$. The annotation may also satisfy $allowsRole(pa, DPO)$, indicating that access to the annotated entity is restricted to the Data Protection Officer role.

4.2 Granularity and Containment

Artifacts are represented at dataset, record, and cell levels (Figure 1). Containment is captured through `containsDR` (dataset–record) and `containsRC` (record–cell). These relations enable analyses that connect dataset-level declarations with record- or cell-level elements, and conversely allow fine-grained reasoning to be summarised at higher abstraction levels.

Annotations may be attached at any granularity. Containment further supports an inheritance principle: when an annotation is attached at a container level, it is interpreted as applying to contained elements unless overridden by a more specific annotation. This mechanism allows governance policies to be expressed concisely while preserving fine-grained exceptions. For artifact types without internal containment structure, such as trained models or evaluation artifacts, annotations apply directly to the artifact instance.

Example. A curated dataset entity d may be annotated with $hasUsage(clinical\ research)$. This usage constraint can be interpreted as applying to records r such that $containsDR(d, r)$, and to cells c such that $containsRC(r, c)$, unless a record- or cell-level annotation specifies a stricter constraint.

4.3 Category Implication

Sensitivity categories form a hierarchy captured through the `implies` relation in Figure 1. This relation supports reasoning at different abstraction levels. If a category k_1 implies k_2, then exposure to k_1 entails exposure to k_2. Queries may therefore be expressed at a desired level of abstraction without enumerating all specialised subcategories.

Example. If *biometric* implies *sensitive_personal_data*, then any artifact exposed to *biometric* is also exposed to *sensitive_personal_data*. This enables auditors to formulate queries using high-level regulatory categories while relying on fine-grained annotations internally.

4.4 Influence vs. Privacy Exposure

We distinguish two notions used by different auditing tasks.

Influence (contribution). Influence captures whether an artifact participated in the production of another artifact independently of privacy semantics. We use provenance reachability over `prov:wasDerivedFrom` edges to determine whether an entity a contributes to an entity b. This notion supports tasks such as right-to-erasure impact analysis, consent auditing, and responsibility attribution.

Privacy exposure. Privacy exposure captures which privacy properties are associated with an artifact after accounting for operator-aware propagation (Section 5). An entity e is exposed to a category k when there exists an annotation pa such that $annotates(pa, e)$ and $hasCategory(pa, k)$ (up to implication closure). Unlike influence, exposure depends on how annotations are assigned, minted, attenuated, and propagated across executed steps. Exposure queries are evaluated relative to the organisational role of the requesting user, as discussed in Section 6.

5 Operator-Aware Annotation Propagation

Section 4 introduced privacy annotations and the ontology fragment shown in Figure 1. This section specifies how annotations evolve during retrospective execution. Rather than defining propagation independently for each operator, we adopt an algebraic formulation inspired by provenance semiring abstractions [**?**].

Propagation is defined independently for each generated entity. A step run produces annotations for every generated entity using only those input entities that effectively contributed to its derivation. Dataset- and record-level annotations are derived separately through the containment relations introduced in Section 4.

Let SR denote a step run consuming entities $In(SR)$ through `used` relations and generating entities $Out(SR)$ through `wasGeneratedBy` relations.

Each entity e is associated with an annotation value

$$\pi(e) = (C(e), U(e), R(e), A(e)),$$

capturing respectively sensitivity categories, authorised usage contexts, retention or erasure obligations, and authorised access roles.

5.1 Operator Classes and Semantics

Each step run is associated with an operator type describing the executed transformation:

$$opType(SR) \in \{\mathsf{RED}, \mathsf{TR}, \mathsf{AGG}, \mathsf{FUS}, \mathsf{LEARN}\}.$$

Operator types determine three aspects of annotation propagation:
(i) the granularity at which contributors are identified, (ii) whether additional annotations may be introduced through minting, and (iii) whether attenuation may apply.

- RED (reduction) operators filter or project subsets of inputs. Contributors correspond only to selected entities.
- TR (transformation) operators derive new feature values from existing entities. Contributors are determined at the level of consumed cells or attributes.
- AGG (aggregation) operators summarise multiple contributors into a single output entity. Aggregation support determines whether attenuation applies.
- FUS (fusion) operators combine entities originating from multiple sources. Fusion may mint additional sensitivity categories when linkage risks arise.
- LEARN (learning) operators produce models or evaluation artifacts from training data. Contributors correspond to training entities or derived features.

The following subsections formalise how contributor annotations are combined, extended through minting, or attenuated.

5.2 Contributor-Based Combination

Annotations propagate only from entities that contributed to the production of a generated entity. For each generated entity $b \in Out(SR)$, let

$$Contrib(SR, b) \subseteq In(SR)$$

denote contributors determined through provenance derivation relations.
Contributor determination relies exclusively on retrospective provenance relations recorded during execution and is therefore deterministic given a fixed execution trace. As such, contributors may exist at different granularities depending on the operator. Feature engineering steps, for instance, generate new feature values represented as cell entities; contributors are therefore determined at cell level from the input cells used during computation. Let

$$\Pi_{in}(b) = \{\pi(a) \mid a \in Contrib(SR, b)\}$$

denote contributor annotations.
The combined annotation is defined as

$$\bigotimes \Pi_{in}(b) = \left(\bigcup C(a), \bigcap U(a), \bigcup R(a), \bigcap A(a)\right).$$

Each component of the annotation value propagates according to governance-oriented combination semantics reflecting distinct privacy objectives. Sensitivity categories accumulate across contributors through union, ensuring that all privacy risks associated with contributing entities remain visible in generated artifacts. Usage constraints become stricter through intersection, guaranteeing that downstream processing remains compliant with all authorised purposes associated

with contributors. Retention or erasure obligations accumulate through union, since any deletion obligation affecting a contributor must remain enforceable for derived artifacts. Finally, authorised access roles propagate conservatively through intersection: a generated entity is accessible only to organisational roles authorised by all contributors, thereby preventing privilege escalation through data combination. This conservative policy prevents sensitive contributors from indirectly broadening access through downstream transformations and reflects a least-privilege governance principle.

Sensitivity categories are interpreted modulo the implication relation introduced in Section 4. After combination, implied categories may be inferred and redundant ancestors removed through canonicalisation. Usage constraints, erasure obligations, and authorised roles are interpreted directly as policy sets and therefore require no implication closure.

These propagation choices ensure that privacy governance remains monotonic with respect to risk and obligation: combining contributors cannot weaken usage restrictions, eliminate deletion duties, or broaden authorised access. It is worth underlining that these combination choices reflect established data governance principles commonly associated with regulatory frameworks such as GDPR. Union-based accumulation preserves accountability by ensuring that risks and deletion obligations originating from any contributor remain traceable in derived artifacts. Intersection-based restriction enforces purpose limitation and least-privilege access by preventing downstream processing or visibility beyond what is authorised for all contributors. Consequently, annotation propagation aligns operational pipeline behaviour with organisational compliance objectives without requiring manual intervention at execution time.

Example. Consider a filtering step that removes Bob's record from a dataset. Generated entities are derived only from retained contributors. Sensitivity categories and usage constraints therefore reflect only the remaining inputs. Erasure obligations associated with Bob do not propagate to outputs because Bob no longer contributes to their derivation. Likewise, access permissions inherited from Bob's data do not influence downstream artifacts once his record has been excluded.

5.3 Annotation Minting

Certain transformations introduce privacy risks that do not originate directly from contributor annotations. Such risks arise from how data are combined or analysed during execution. We capture these effects through a minting function

$$Mint(SR, b),$$

which associates additional annotation components with a generated entity b.

Minting rules are defined declaratively as part of operator semantics or governance policies rather than through manual annotation during execution. Such policies are assumed to be defined prior to execution as part of organisational governance configuration and remain independent of individual pipeline runs.

They rely on semantic annotations associated with input entities, in particular cell-level annotations describing attribute roles or data semantics.

Attribute semantics are represented using the sensitivity categories introduced in Section 4, including categories such as *identifier*, *quasi_ identifier*, *biometric*, or *health*. These annotations may originate from governance metadata, domain ontologies, or schema registration processes. Minting evaluates operator configurations together with contributor annotations.

Example. Consider a fusion step joining a clinical dataset with demographic information using a postal code attribute. Cells corresponding to the postal code are annotated as *quasi_ identifier*. Although neither dataset alone enables identification, joining them increases linkage risk. A platform policy therefore specifies that joins performed on quasi-identifiers mint an additional sensitivity category *reidentification_ risk*. The system automatically derives

$$Mint(SR, b) = \{reidentification\_\ risk\}.$$

Learning operators may similarly mint exposure categories when models are trained using personal or sensitive data.

Minting may also introduce additional access restrictions when governance policies associate specific risks with restricted organisational responsibilities. For example, a fusion step that introduces re-identification risk may additionally restrict access to the *DPO* role. Minting therefore captures privacy properties introduced by transformation semantics rather than by direct propagation from contributors.

5.4 Attenuation

Sensitivity categories differ from usage constraints, erasure obligations, and access permissions in that they characterise exposure risk rather than regulatory or organisational commitments. While obligations and permissions must propagate conservatively to preserve governance guarantees, exposure risk may legitimately decrease when transformations reduce identifiability through summarisation or abstraction. Indeed, certain transformations reduce privacy exposure by summarising information across multiple contributors. Aggregation operators may therefore attenuate sensitivity categories associated with generated entities.

Attenuation applies exclusively to sensitivity categories. Usage constraints, erasure obligations, and authorised roles remain monotonic even when sensitivity decreases. These components encode regulatory obligations and organisational permissions that cannot be weakened solely through aggregation semantics. Let k denote the aggregation support associated with a generated entity b, i.e., the number of contributor entities participating in its computation. For aggregation operators, attenuation is defined as

$$Attenuate_{SR}(\pi) = attenuate_k(\pi) \quad \text{if } opType(SR) = \mathsf{AGG}.$$

The attenuation function depends on a governance parameter k_{min}. When $k \geq k_{min}$, identifier-related sensitivity categories may be removed, while domain-sensitive categories remain.

The value of k_{min} is defined through governance or regulatory policies associated with the execution environment or dataset rather than individual pipeline executions.

Example. Consider a step computing the average heart rate per hospital from individual patient records. Each input record carries a *biometric* sensitivity category. If the aggregation combines a sufficiently large number of contributors ($k \geq k_{min}$), direct identification risk decreases. Identifier-related categories may therefore be attenuated, while health-related sensitivity remains associated with the aggregated result.

Attenuation captures reduced identifiability resulting from summarisation. It does not claim formal anonymisation guarantees and remains compatible with recomputation obligations when contributors request deletion. Attenuation therefore represents a governance abstraction reflecting reduced exposure rather than a formal privacy guarantee.

5.5 Operator Propagation Semantics

Operator types determine contributor selection together with minting and attenuation behaviour. Reduction and transformation operators primarily propagate contributor annotations. Aggregation operators may attenuate identifier-related sensitivity depending on aggregation support. Fusion and learning operators may introduce additional risks and governance restrictions through minting policies. Together, these mechanisms determine privacy exposure as defined in Section 4.

Propagation therefore determines which privacy properties become associated with derived artifacts as a consequence of execution semantics. Access control complements this process by determining which portions of provenance and exposure information may be observed by different organisational roles during auditing. Section 6 builds upon these propagation results to define role-aware privacy queries over authorised provenance views.

6 Privacy Auditing Queries

Sections 4 and 5 introduced the privacy ontology fragment and operator-aware propagation semantics illustrated in Figure 1. Together, these representations enable privacy auditing questions to be expressed as analyses over retrospective lifecycle provenance.

Let $G = (V, E)$ denote the retrospective provenance graph composed of entities and step runs connected through PROV relations. Queries exploit both contribution dependency and propagated privacy annotations. These two reasoning modes support complementary auditing tasks. Query answering is evaluated relative to the organisational role of the requesting user. Let $role(q)$ denote the role associated with a query request q. Access permissions propagated as part

of annotation semantics (Section 5) determine an authorised provenance view visible to the requester. An entity e is visible to a query q iff

$$role(q) \in A(e),$$

where $A(e)$ denotes the authorised access roles associated with e. A provenance relation is visible only when both endpoints are visible, preventing indirect disclosure through structural dependencies. The following query classes operate over this authorised provenance view.

6.1 Contribution-Based Queries

Contribution-based queries rely on provenance reachability over `prov:wasDerivedFrom`. They capture influence relationships independently of privacy exposure semantics but remain constrained by authorised visibility.

Right to erasure. Deletion requests require identifying downstream artifacts influenced by specific records. Following Bob's deletion request, the system identifies datasets, trained models, and evaluation artifacts that depend on his physiological record within the authorised provenance view. Restricted entities remain hidden when access permissions do not permit inspection.

Consent compliance. Consent auditing verifies that data usage respects authorised contexts. For example, Bob authorises the use of his data for clinical research only. A pipeline run associated with commercial product development can be detected as non-compliant when his record contributes to the execution and is visible to the requesting role.

Accountability. Accountability queries determine which data influenced a given artifact. An auditor may request the list of records and datasets contributing to a deployed prediction model in order to understand data dependencies. Depending on the requesting role, answers may be returned at different granularities, for example dataset-level lineage for data scientists and record-level lineage for auditors or data protection officers.

These queries rely solely on contribution dependency and do not require annotation propagation.

6.2 Exposure-Based Queries

Exposure-based queries rely on privacy annotations attached to entities through operator-aware propagation (Section 5). Exposure therefore reflects privacy properties resulting from transformations rather than simple reachability.

Sensitivity exposure. Exposure analysis determines whether an artifact carries a given sensitivity category. For example, a clinician may ask whether a deployed model is exposed to biometric information derived from heart-rate measurements, even when the original attribute is no longer explicitly present. Category implication relationships allow queries to be expressed at different abstraction levels while respecting authorised visibility.

Transformation responsibility. Responsibility queries identify executions that introduced or propagated privacy exposure. For example, if a *reidentification_risk* category is minted during a dataset fusion step, the corresponding join execution can be identified as responsible for introducing the risk, provided it is accessible to the requesting role.

Data minimisation. Data minimisation analysis examines whether sensitive information contributed to model production. A data scientist may determine whether biometric measurements propagated through engineered features to influence a trained model despite alternative non-sensitive inputs being available. Restricted contributors may be abstracted or omitted depending on access permissions.

Contribution-based queries capture influence relationships required for erasure, consent auditing, and accountability. Exposure-based queries analyse privacy properties resulting from annotation propagation and ontology reasoning. Role-aware access control complements these reasoning modes by determining which portions of provenance and exposure information are observable to different organisational roles. Together, they support systematic and governance-aware auditing across the machine learning lifecycle.

The next section describes the system architecture supporting execution of these queries across lifecycle provenance graphs, and show evaluation exercises in the context of use cases....

7 Experimental Evaluation

8 Related Work

This section surveys existing work related to provenance management, privacy governance, and auditing in machine-learning pipelines. We organise prior research into five complementary categories.

8.1 Provenance in Machine-Learning Pipelines

- Schlegel et al., *"End-to-End Provenance Capture for Machine Learning Pipelines"*, Information Systems, 2025. (PROV-compliant lineage extraction from MLflow and software repositories; focuses on reproducibility rather than privacy governance.)
- Zaharia et al., *"MLflow: A Platform for Managing the Machine Learning Lifecycle"*, IEEE Data Engineering Bulletin, 2018. (Experiment tracking and lineage support widely adopted in ML engineering practice.)
- Kubeflow Documentation, *Model Registry and Lineage Tracking*, Kubeflow Project. (Industrial lineage tooling motivating lifecycle provenance needs.)
- Garijo and Gil, *"Augmenting PROV with Plans in P-Plan"*, CEUR Workshop Proceedings, 2012. (Prospective provenance model for workflows and plans.)

- Chapman et al., *"Whole Tale: Capture, Management, and Publication of Reproducible Computational Research"*, IEEE eScience / PEARC ecosystem, 2018–2021. (Provenance-aware infrastructure supporting executable research objects and lifecycle reproducibility in data science workflows.)
- Chard, Gaffney, Foster, et al., *"I'll Take That to Go: Big Data Bags and Minimal Identifiers for Exchange of Large, Complex Datasets"*, IEEE BigData, 2016. (Data packaging and provenance capture mechanisms supporting reproducible computational experiments across distributed environments.)
- Pimentel, Murta, Braganholo, and Freire, *"A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks"*, MSR (Mining Software Repositories), 2019. (Analysis of reproducibility challenges in notebook-based data science workflows motivating systematic provenance capture.)

8.2 Privacy Ontologies and GDPR Compliance

- Pandit and Lewis, *"Modelling Provenance for GDPR Compliance using Linked Open Data Vocabularies"*, PrivOn Workshop (ISWC), 2017. (GDPRov ontology extending PROV-O and P-Plan for consent and data lifecycle provenance.) [**?**] *Key GDPR provenance ontology.* [**?**]
- Pandit, O'Sullivan, and Lewis, *"Queryable Provenance Metadata for GDPR Compliance"*, SEMANTiCS Conference, Elsevier Procedia Computer Science, 2018. (SPARQL querying of GDPR compliance metadata using GDPRov and GDPRtEXT.) [**?**]
- Esteves et al., *"Analysis of Ontologies and Policy Languages to Represent GDPR Concepts"*, Semantic Web Journal. (Survey of privacy ontologies and regulatory modelling approaches.) [**?**]
- GDPRtEXT Consortium, Linked Data GDPR vocabulary. (Regulatory terminology representation rather than execution semantics.)

8.3 Provenance-Based Privacy Auditing

- Pandit et al., *Queryable GDPR Compliance via Provenance Metadata*, SEMANTiCS 2018. (Provenance queries supporting GDPR readiness documentation.) [**?**]
- Campagna et al., *"Achieving GDPR Compliance through Provenance"*, SBBD Workshop. (Compliance reasoning over provenance traces.) [**?**]
- Early provenance auditing literature:
  Buneman et al., *"Why and Where: A Characterization of Data Provenance"*, ICDT 2001.
  (Provenance reachability foundations supporting accountability reasoning.)

8.4 Policy Propagation and Sticky Policies

- Thion et al., *"Provenance-Based Enforcement of Purpose Limitation"*, ACM SACMAT / SEC-SAC style policy enforcement work. (Provenance-driven policy propagation.)

- Pearson et al., *Sticky Policies for Data Governance*, IBM Research reports. (Policies travelling with data across transformations.)
- Information-flow control literature:
  Sabelfeld and Myers, *"Language-Based Information-Flow Security"*, IEEE Journal on Selected Areas in Communications, 2003. (Foundations for policy propagation semantics.)

8.5 Access Control and Abstraction for Provenance Graphs

- Cadenhead et al., *"A Language for Provenance Access Control"*, CODASPY, ACM, 2011. (Path-based access policies for provenance DAGs.) [**?**]
- Danger et al., *"Access Control and View Generation for Provenance Graphs"*, Future Generation Computer Systems, 2015. (Role-based provenance disclosure in healthcare workflows.) [**?**]
- Danger et al., *"Access Control for OPM Provenance Graphs"*, IPAW 2012, LNCS. (Access control policies over provenance structures.) [**?**]
- Missier et al., *"ProvAbs: Model, Policy and Tooling for Abstracting PROV Graphs"*, TAPP / arXiv 2014. (Provenance abstraction and redaction policies.) [**?**]
- Fan et al., *"PACLP: Fine-Grained Partition-Based Access Control for Provenance"*, arXiv 2019. (Segmentation-based provenance access control.) [**?**]
- Fan et al., *"Fine-Grained Provenance-Based Policy Algebra"*, arXiv 2020. (Policy algebra for provenance-aware access decisions.) [**?**]

9 Conclusions

References